

Revisão de Android

Compose, ContentProvider, SharedPreferences, Lifecycle,
Coroutines, MVVM e StateFlow/LiveData

Aplicação de exemplo: `ComposeReview`

Pedro Oliveira

3 de julho de 2026

Sumário

1	Introdução	2
2	Estrutura do projeto	2
3	Parte I — Jetpack Compose	3
3.1	Os seis conceitos, um a um	3
3.1.1	<code>@Composable</code>	3
3.1.2	Composição e recomposição	3
3.1.3	<code>state</code>	4
3.1.4	<code>remember</code>	5
3.1.5	<code>mutableStateOf</code>	5
3.1.6	UI declarativa	6
3.2	Fluxo de dados: do toque na tela até a recomposição	6
4	Parte II — arquitetura, dados e ciclo de vida	7
4.1	Content Provider e compartilhamento de dados	7
4.2	SharedPreferences e suas limitações	8
4.3	Arquitetura MVVM	9
4.4	Coroutines	10
4.5	StateFlow / LiveData	11
4.6	Activity/Fragment lifecycle	12
5	Fluxo de dados da tela Notas: do toque em "Add" até a UI atualizada	13
6	Conclusão	13

1 Introdução

Este documento acompanha o projeto Android `jetpack-compose-review/`, uma aplicação de exemplo que revisa, na prática, sete tópicos que costumam cair em entrevista/prova de Android:

- Jetpack Compose: `@Composable`, composição e recomposição, `state`, `remember`, `mutableStateOf` e UI declarativa;
- Content Provider e compartilhamento de dados;
- SharedPreferences e suas limitações;
- Activity/Fragment lifecycle;
- Coroutines;
- MVVM;
- StateFlow / LiveData.

O projeto está dividido em duas partes. A **Parte I** (Seção 3) cobre só Jetpack Compose puro, com um contador e uma lista de tarefas cujo estado vive inteiramente na função `@Composable` — a versão mais simples possível dos conceitos de Compose. A **Parte II** (Seções 4.1 a 4.5) adiciona uma terceira tela, *Notas*, que já não guarda estado só localmente: ela persiste dados em SQLite por trás de um `ContentProvider`, guarda preferências simples em `SharedPreferences`, organiza a lógica em camadas MVVM, usa `Coroutines` para I/O assíncrono e expõe o estado da tela via `StateFlow` (comparado lado a lado com `LivData`). Uma quarta tela, separada e escrita com Views/XML clássicos, existe só para tornar visível o `Activity/Fragment lifecycle` completo, algo que não aparece em um app 100% Compose de Activity única.

2 Estrutura do projeto

O projeto segue a estrutura padrão de um módulo Android/Gradle:

```
jetpack-compose-review/
|-- build.gradle.kts          # configuracao raiz (plugins)
|-- settings.gradle.kts      # declara o modulo :app
|-- app/
|   |-- build.gradle.kts      # dependencias (Compose, lifecycle,
|   |                          # coroutines, appcompat/fragment)
|   |-- src/main/
|       |-- AndroidManifest.xml # registra Activities e o Provider
|       |-- java/com/pedro/composereview/
|           |-- MainActivity.kt  # Activity Compose + telas 1 e 2
|           |-- data/           # Parte II: ContentProvider
|               |-- Note.kt
|               |-- NotesContract.kt
|               |-- NotesDbHelper.kt      (SQLiteOpenHelper)
|               |-- NotesProvider.kt      (ContentProvider)
|               |-- NotesRepository.kt    (Coroutines + Flow)
|               |-- prefs/
|                   |-- UserPreferences.kt (SharedPreferences)
|                   |-- ui/
|                       |-- notes/
```

```

|         | | | | | -- NotesViewModel.kt (MVVM + StateFlow/LiveData)
|         | | | | | '-- NotesScreen.kt    (tela 3, Compose)
|         | | | | | '-- theme/
|         | | | | |     |-- Color.kt
|         | | | | |     |-- Theme.kt
|         | | | | |     '-- Type.kt
|         | | | | | '-- lifecycle/          # Parte II: demo classica
|         | | | | |     |-- LifecycleDemoActivity.kt
|         | | | | |     '-- LifecycleDemoFragment.kt
|         |-- res/
|         | |-- layout/                    # XML só das telas de lifecycle
|         | | |-- activity_lifecycle_demo.xml
|         | | '-- fragment_lifecycle_demo.xml
|         |-- values/
|         | |-- strings.xml
|         | '-- themes.xml
|-- docs/
    '-- revisao-jetpack-compose.tex    # este documento

```

A regra geral do projeto: tudo que é tela normal do app é Compose puro (`MainActivity.kt` e o pacote `ui/`); `res/layout/` e XML existem *apenas* na demonstração de lifecycle clássico, isolada no pacote `lifecycle/`, para não misturar os dois estilos na mesma tela.

3 Parte I — Jetpack Compose

3.1 Os seis conceitos, um a um

3.1.1 @Composable

`@Composable` é a anotação que transforma uma função Kotlin comum em uma **unidade de UI** que o compilador do Compose sabe como rastrear, (re)executar e otimizar. Uma função marcada com `@Composable` pode chamar outras funções `@Composable`, ler estado (`State<T>`) e emitir elementos de UI — mas não pode ser chamada de código Kotlin comum fora desse contexto.

No projeto, todas as telas são funções `@Composable`: `ReviewApp`, `SectionTitle`, `CounterCard`, `TodoCard`, `TaskList` e `TaskRow`. Exemplo:

```

1 @Composable
2 private fun SectionTitle(text: String) {
3     Text(
4         text = text,
5         fontWeight = FontWeight.Bold,
6         style = MaterialTheme.typography.titleMedium
7     )
8 }

```

Listing 1: Uma função `@Composable` simples e reutilizável

Note que `SectionTitle` não devolve uma `View`: ela apenas *descreve* que, naquele ponto da árvore, deve existir um `Text` com determinado estilo. Quem decide *quando* e *como* desenhar isso é o runtime do Compose — esse é o cerne da UI declarativa (Seção 3.1.6).

3.1.2 Composição e recomposição

Composição é o processo de executar as funções `@Composable` e montar a árvore de UI resultante (quais elementos existem, com quais parâmetros, aninhados de que forma). Isso acontece na

primeira renderização da tela.

Recomposição é o processo de executar *de novo* apenas as funções `@Composable` cujas leituras de estado mudaram, para atualizar a árvore — sem recriar a `Activity` nem reconstruir a UI do zero. O Compose rastreia, para cada composável, quais objetos de `State` ele leu durante a composição; quando um desses valores muda, apenas os composáveis que o leram (e, quando necessário, seus descendentes) são recompostos.

Para tornar esse processo visível, `CounterCard` mantém um contador de recomposições (`recompositions`) que *não* é um `State` — é uma variável comum guardada com `remember`. Ela é incrementada toda vez que a função roda de novo, mas sua leitura não "assina" a composição atual (diferente de um `mutableStateOf`), então ela não causa recomposições adicionais por conta própria — apenas reflete quantas vezes o `count` mudou até agora:

```

1 @Composable
2 fun CounterCard() {
3     var count by remember { mutableStateOf(0) }
4     val recompositions = remember { intArrayOf(0) }
5     recompositions[0] += 1
6
7     Card(/* ... */) {
8         Column(/* ... */) {
9             Text("Valor: $count")
10            Text("Esta secao recompos ${recompositions[0]} vez(es)")
11            Row {
12                Button(onClick = { count++ }) { Text("+1") }
13                Button(onClick = { count-- }) { Text("-1") }
14                TextButton(onClick = { count = 0 }) { Text("Resetar") }
15            }
16        }
17    }
18 }

```

Listing 2: Tornando a recomposição visível na tela

A cada toque em "+1", "-1" ou "Resetar": (1) `count` muda; (2) o Compose marca `CounterCard` como inválido, pois foi ele quem leu `count` durante a composição; (3) `CounterCard` é executado de novo (recomposição); (4) `recompositions[0]` é incrementado e o novo valor aparece na tela — evidenciando que a recomposição de fato ocorreu.

3.1.3 state

Em Compose, `state` (estado) é qualquer valor que, ao mudar, deveria fazer a UI se atualizar. Ele é representado pela interface `State<T>` (e sua variante mutável, `MutableState<T>`). O Compose observa leituras de `State` feitas dentro de um escopo de composição e usa isso para saber o que recompor quando o valor muda.

O projeto usa estado em dois lugares com naturezas diferentes:

- **Estado local e primitivo:** `count` (um `Int`) em `CounterCard` e `input` (uma `String`) em `TodoCard`.
- **Estado de coleção:** `tasks`, uma `SnapshotStateList<String>` criada com `mutableStateListOf(...)`, que notifica a composição quando itens são adicionados ou removidos (`tasks.add(...)`, `tasks.remove(...)`), sem precisar reatribuir a variável inteira.

Também é demonstrado o padrão de **state hoisting**: em vez de cada composável filho guardar seu próprio estado, `TodoCard` concentra o estado (`input` e `tasks`) e o repassa como parâmetros e callbacks para `TaskList` e `TaskRow`, que ficam *stateless* — apenas recebem dados e informam eventos para cima:

```

1 @Composable
2 fun TaskList(tasks: List<String>, onRemove: (String) -> Unit) {
3     Column {
4         tasks.forEach { task ->
5             TaskRow(task = task, onRemove = { onRemove(task) })
6         }
7     }
8 }
9
10 @Composable
11 fun TaskRow(task: String, onRemove: () -> Unit) {
12     Row {
13         Text(text = task, modifier = Modifier.weight(1f))
14         TextButton(onClick = onRemove) { Text("Remover") }
15     }
16 }

```

Listing 3: State hoisting: pai com estado, filhos sem estado

Essa separação torna `TaskList` e `TaskRow` fáceis de reutilizar, testar e pré-visualizar isoladamente, porque não dependem de *onde* o estado mora — apenas do que recebem por parâmetro.

3.1.4 remember

Funções `@Composable` podem ser executadas várias vezes (recompostas). Sem alguma forma de persistir valores entre essas execuções, qualquer variável local seria recriada do zero a cada recomposição — um contador sempre voltaria a zero, por exemplo.

`remember` resolve isso: ele guarda um valor na memória interna da composição (a *slot table*) e o devolve nas recomposições seguintes, em vez de recalculá-lo. O valor só é recriado se o composável sair completamente da árvore (por exemplo, some da tela) e entrar de novo, ou se as chaves passadas a variações como `remember(key)` mudarem.

No projeto, `remember` aparece em toda inicialização de estado:

```

1 var count by remember { mutableStateOf(0) }
2 val recompositions = remember { intArrayOf(0) }
3 var input by remember { mutableStateOf("") }
4 val tasks = remember { mutableStateListOf("Revisar @Composable", "Estudar remember") }

```

Listing 4: remember preservando estado entre recomposições

Em todos os casos, a lambda passada para `remember` só é executada **uma vez** (na primeira composição); nas recomposições seguintes, `remember` simplesmente devolve o valor já existente.

3.1.5 mutableStateOf

`mutableStateOf(valorInicial)` cria um objeto `MutableState<T>` observável: sempre que sua propriedade `.value` é reatribuída, o Compose sabe que precisa invalidar (e depois recompor) qualquer composável que tenha lido esse valor durante a composição.

Combinado com o delegate de propriedade `by` (do pacote `androidx.compose.runtime`, via `getValue/setValue`), é possível tratar o estado quase como uma variável comum:

```

1 var count by remember { mutableStateOf(0) }
2 // equivalente, sem o delegate 'by':
3 // val countState = remember { mutableStateOf(0) }
4 // countState.value++
5
6 count++ // lê e escreve em countState.value internamente

```

Listing 5: mutableStateOf com o delegate 'by'

O mesmo padrão é usado para `input` em `TodoCard` (`var input by remember { mutableStateOf("") }`), atualizado a cada tecla digitada via `onValueChange = { input = it }` do `OutlinedTextField`. Para a lista de tarefas, o equivalente para coleções é usado: `mutableStateListOf(...)`, que devolve uma lista cujas operações de mutação (`add`, `remove`) já disparam recomposição sozinhas.

3.1.6 UI declarativa

Em UI **imperativa** (o modelo clássico de `Views` no Android), o desenvolvedor descreve *passo a passo como transformar* a UI de um estado para outro: cria uma `View`, chama `findViewById`, depois `setText(...)`, `setVisibility(...)`, etc., sempre que algo muda.

Em UI **declarativa** (o modelo do Compose), o desenvolvedor descreve apenas *como a UI deve ser para um dado estado atual* — uma função pura, em essência: `UI = f(state)`. Quando o estado muda, a função é executada de novo (recomposição) e o Compose calcula sozinho a diferença necessária na árvore de UI, sem que o código precise dizer "esconda este botão" ou "atualize aquele texto" manualmente.

Isso fica evidente em `CounterCard`: o texto `"Valor: $count"` não é atualizado com um comando imperativo do tipo `textView.setText(...)`. Em vez disso, a função inteira é reexecutada com o novo valor de `count`, e a nova string `"Valor: $count"` é simplesmente o resultado dessa execução:

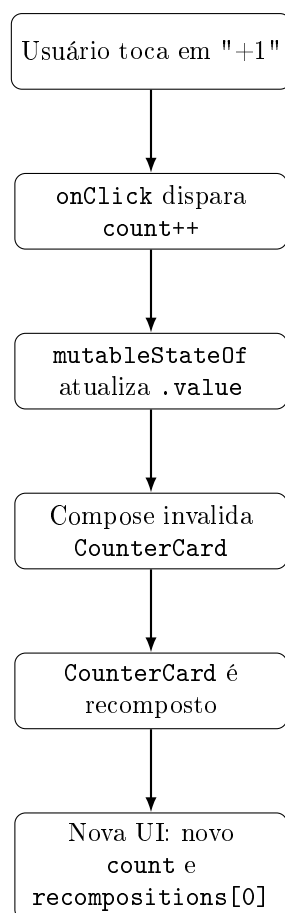
```
1 Text("Valor: $count", style = MaterialTheme.typography.headlineSmall)
```

Listing 6: `UI = f(state)`: o texto é apenas função do estado atual

Não existe, em nenhum lugar do código, uma chamada explícita para "atualizar o texto" — o Compose garante isso automaticamente porque `Text` leu `count` durante a composição.

3.2 Fluxo de dados: do toque na tela até a recomposição

O diagrama abaixo resume o ciclo que ocorre a cada toque no botão `"+1"` do contador:



O mesmo ciclo se repete para a lista de tarefas, com a diferença de que o estado (**tasks**) mora em **ToDoCard** e é apenas *lido* por **TaskList/TaskRow** — por isso, ao adicionar ou remover uma tarefa, é **ToDoCard** (e os composables que efetivamente leem **tasks**) que recompõe, não a árvore inteira do aplicativo.

4 Parte II — arquitetura, dados e ciclo de vida

A Parte I usou estado 100% local: quando **MainActivity** morre (app fechado, processo do sistema encerrado por falta de memória), o contador e as tarefas somem. A Parte II resolve isso com a tela *Notas*, construída em camadas — exatamente o tipo de arquitetura usada em apps reais — e introduz os outros cinco tópicos revisados.

4.1 Content Provider e compartilhamento de dados

ContentProvider é um dos quatro componentes de app do Android (junto com **Activity**, **Service** e **BroadcastReceiver**). Sua função é expor dados por trás de um contrato de **URIs**, para que outros componentes — ou até outros apps, quando **exported="true"** — leiam e escrevam sem conhecer o mecanismo de armazenamento real (SQLite, um arquivo, uma API remota, etc.). É o mesmo princípio usado pelo Android para os provedores nativos de Contatos, Calendário e Mídia.

No projeto, **NotesProvider** expõe a tabela SQLite **notes** (criada por **NotesDbHelper**) através de duas URIs — a coleção inteira e um item específico — resolvidas com um **UriMatcher**:

```
1 class NotesProvider : ContentProvider() {  
2  
3     private lateinit var dbHelper: NotesDbHelper  
4 }
```

```

5  override fun onCreate(): Boolean {
6      dbHelper = NotesDbHelper(context!!)
7      return true
8  }
9
10 override fun query(
11     uri: Uri, projection: Array<out String>?, selection: String?,
12     selectionArgs: Array<out String>?, sortOrder: String?
13 ): Cursor {
14     val db = dbHelper.readableDatabase
15     val cursor = when (uriMatcher.match(uri)) {
16         NOTES -> db.query(TABLE_NAME, projection, selection, selectionArgs,
17             null, null, sortOrder ?: "created_at DESC")
18         NOTE_ID -> db.query(TABLE_NAME, projection, "_id = ?",
19             arrayOf(uri.lastPathSegment), null, null, null)
20         else -> throw IllegalArgumentException("URI desconhecida: $uri")
21     }
22     cursor.setNotificationUri(context!!.contentResolver, uri)
23     return cursor
24 }
25
26 override fun insert(uri: Uri, values: ContentValues?): Uri {
27     val db = dbHelper.writableDatabase
28     val id = db.insert(TABLE_NAME, null, values)
29     val itemUri = ContentUris.withAppendedId(NotesContract.CONTENT_URI, id)
30     context!!.contentResolver.notifyChange(itemUri, null)
31     return itemUri
32 }
33
34 // update(), delete() e getType() seguem o mesmo padrao.
35 }

```

Listing 7: NotesProvider: query e insert via URI, com notifyChange

Dois detalhes merecem destaque:

- `cursor.setNotificationUri(...)` e `contentResolver.notifyChange(...)` são o que permite que *qualquer* componente que tenha registrado um `ContentObserver` naquela URI seja avisado quando os dados mudam — inclusive se a escrita veio de outro processo. É essa notificação que `NotesRepository` usa para transformar mudanças no banco em um Flow reativo (Seção 4.4).
- `NotesProvider` está declarado no `AndroidManifest.xml` com `exported="false"`: só este app pode acessá-lo. Compartilhar de fato com outros apps seria trocar para `exported="true"` e, tipicamente, adicionar `readPermission/writePermission` para controlar quem pode ler e quem pode escrever.

Na prática, o resto do app nunca importa `NotesDbHelper` diretamente — só fala com `ContentResolver` usando `NotesContract.CONTENT_URI`, o que deixa a implementação de armazenamento livre para mudar sem afetar quem consome os dados.

4.2 SharedPreferences e suas limitações

`SharedPreferences` guarda pares chave/valor simples em um arquivo XML privado do app — ideal para preferências pequenas (um nome de usuário, um contador, uma flag de "já mostrou o tutorial"). No projeto, `UserPreferences` envolve esse acesso:

```

1 class UserPreferences(context: Context) {
2     private val prefs: SharedPreferences =
3         context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE)
4 }

```



```
5 fun getUsername(): String = prefs.getString(KEY_USER_NAME, "") ?: ""
6
7 fun setUsername(name: String) {
8     prefs.edit().putString(KEY_USER_NAME, name).apply()
9 }
10
11 fun incrementNotesCreatedTotal() {
12     prefs.edit().putInt(KEY_NOTES_CREATED, getNotesCreatedTotal() + 1).apply()
13 }
14 }
```

Listing 8: UserPreferences: leitura/escrita simples com apply()

No app, `NotesViewModel` usa essa classe para lembrar o nome digitado pelo usuário e o total histórico de notas criadas — valores que sobrevivem mesmo que o processo do app seja encerrado, ao contrário do `remember` da Parte I.

`apply()` grava em disco de forma assíncrona (não bloqueia quem chamou, mas também não confirma que a escrita já terminou); `commit()` é a alternativa síncrona, que bloqueia a thread chamadora até gravar e devolve um `Boolean` de sucesso — por isso `commit()` nunca deveria ser chamado na thread principal.

Limitações que tornam `SharedPreferences` inadequado para além de configurações simples:

- Chaves são `Strings` soltas — sem verificação do compilador, é fácil digitar errado ("`user_name`" vs "`username`") sem nenhum aviso.
- Não existe consulta, filtro, relação entre chaves ou schema — é só um dicionário chave/valor, não um banco de dados; dados estruturados ou relacionais pertencem ao `SQLite/Content-Provider` (Seção 4.1), não aqui.
- O arquivo XML inteiro é carregado para a memória na primeira leitura; arquivos grandes custam I/O logo na inicialização do app.
- Não é observável de forma nativa por `Compose/Flow` — só existe um `OnSharedPreferenceChangeListener` manual baseado em callback, então não serve como fonte de verdade reativa (comparar com `StateFlow`, Seção 4.5).
- Sem suporte a migração de schema — mudar o formato de um valor salvo é responsabilidade inteiramente do código do app.

Para os casos em que essas limitações pesam — dados maiores, necessidade de observar mudanças de forma reativa, ou tipagem mais forte — o Android recomenda hoje o **Jetpack DataStore** (Preferences ou Proto), que expõe os valores como `Flow` em vez de leitura síncrona.

4.3 Arquitetura MVVM

MVVM (*Model-View-ViewModel*) separa a tela em três papéis:

- **Model**: os dados e as regras de acesso a eles — aqui, `Note`, `NotesRepository` e `UserPreferences`.
- **ViewModel**: dona do estado de UI e da lógica de "o que fazer quando o usuário faz X" — aqui, `NotesViewModel`. Sobrevive a mudanças de configuração (rotação de tela) porque seu ciclo de vida é gerenciado pelo Android Jetpack, não pela `Activity/Composable`.
- **View**: só renderiza o estado atual e repassa eventos do usuário para o `ViewModel` — aqui, o composable `NotesCard` (Seção 4.5).

```

1 class NotesViewModel(application: Application) : AndroidViewModel(application) {
2
3     private val repository = NotesRepository(application.contentResolver)
4     private val preferences = UserPreferences(application)
5     private val _uiState = MutableStateFlow(NotesUiState())
6     val uiState: StateFlow<NotesUiState> = _uiState.asStateFlow()
7
8     init {
9         viewModelScope.launch {
10             repository.observeNotes().collect { notes ->
11                 _uiState.update { it.copy(notes = notes, isLoading = false) }
12             }
13         }
14     }
15
16     fun addNote(title: String) {
17         if (title.isBlank()) return
18         viewModelScope.launch {
19             repository.addNote(title.trim())
20             preferences.incrementNotesCreatedTotal()
21         }
22     }
23
24     fun deleteNote(id: Long) {
25         viewModelScope.launch { repository.deleteNote(id) }
26     }
27 }

```

Listing 9: NotesViewModel: estado + eventos, sem nenhuma referência a Compose

A propriedade central de MVVM aqui é a **direção única de dependência**: `NotesViewModel` não importa nada de `androidx.compose`, e `NotesCard` não sabe nada sobre `ContentResolver` ou `SQLite` — ela só lê `uiState` e chama `addNote/deleteNote`. Isso torna o ViewModel testável sem precisar renderizar UI, e a UI substituível (poderia virar uma tela com Views/XML amanhã) sem tocar na lógica de negócio.

4.4 Coroutines

Toda operação de I/O no Android (disco, banco, rede) deveria rodar fora da *main thread*, sob pena de travar a UI (e, em casos extremos, disparar um ANR — *Application Not Responding*). Coroutines Kotlin resolvem isso com funções `suspend` que "pausam" sem bloquear a thread, mais um `CoroutineScope` que controla quando esse trabalho começa e é cancelado.

`NotesRepository` usa duas técnicas de coroutines:

```

1 suspend fun addNote(title: String): Unit = withContext(Dispatchers.IO) {
2     val values = ContentValues().apply {
3         put(NotesContract.Columns.TITLE, title)
4         put(NotesContract.Columns.CREATED_AT, System.currentTimeMillis())
5     }
6     contentResolver.insert(NotesContract.CONTENT_URI, values)
7     Unit
8 }
9
10 fun observeNotes(): Flow<List<Note>> = callbackFlow {
11     val observer = object : ContentObserver(Handler(Looper.getMainLooper())) {
12         override fun onChange(selfChange: Boolean) { trySend(queryNotes()) }
13     }
14     contentResolver.registerContentObserver(NotesContract.CONTENT_URI, true, observer)
15     trySend(queryNotes())
16     awaitClose { contentResolver.unregisterContentObserver(observer) }
17 }.flowOn(Dispatchers.IO)

```

Listing 10: `withContext` para offload de I/O e `callbackFlow` para observar mudanças

- `withContext(Dispatchers.IO)` move o bloco para uma thread pool otimizada para I/O e devolve o resultado quando termina — usado em `addNote/deleteNote`, que fazem uma escrita pontual.
- `callbackFlow` faz a ponte entre o mundo de callbacks do Android (`ContentObserver.onChange`) e o mundo de coroutines (`Flow`): cada mudança nos dados chama `trySend(...)`, emitindo uma nova lista para quem estiver coletando o `Flow`. `awaitClose` garante que o `ContentObserver` é removido quando ninguém mais coleta, evitando vazamento.

Quem inicia essas coroutines é sempre o `ViewModel`, com `viewModelScope.launch { ... }` (Seção 4.3): esse escopo é cancelado automaticamente em `onCleared()`, então uma coroutine nunca continua rodando depois que a tela que a originou não existe mais.

4.5 StateFlow / LiveData

Ambos são contêineres **observáveis** — a UI se inscreve e é notificada quando o valor muda — mas com origens e comportamento diferentes:

- **LiveData** é mais antigo (pré-Coroutines/Flow), é *lifecycle-aware* por padrão (só entrega valores para observadores `STARTED` ou mais) e é observado com `observe(lifecycleOwner) { ... }` fora do Compose, ou `observeAsState()` dentro dele.
- **StateFlow** é construído sobre Coroutines/Flow, sempre tem um valor atual (não existe "ainda não emituiu nada"), suporta os operadores de Flow (`map`, `combine`, `debounce`, ...) e é a opção recomendada atualmente para expor estado de um `ViewModel`.

Para comparar os dois lado a lado, `NotesViewModel` expõe a mesma informação (quantidade de notas) das duas formas — a `LiveData` é derivada do próprio `StateFlow` com `asLiveData()`:

```
1 val uiState: StateFlow<NotesUiState> = _uiState.asStateFlow()
2
3 val noteCountLiveData: LiveData<Int> = uiState
4   .map { it.notes.size }
5   .asLiveData()
```

Listing 11: A mesma informacao exposta como `StateFlow` e como `LiveData`

E consumidas na tela (`NotesCard`) com o equivalente Compose de cada uma:

```
1 val uiState by viewModel.uiState.collectAsStateWithLifecycle()
2 val liveNoteCount by viewModel.noteCountLiveData.observeAsState(initial = 0)
```

Listing 12: Coletando `StateFlow` e observando `LiveData` a partir do Compose

`collectAsStateWithLifecycle()` só mantém a coleta ativa enquanto a tela está pelo menos `STARTED`, evitando trabalho desperdiçado em background — o mesmo comportamento *lifecycle-aware* que a `LiveData` já tinha nativamente.

4.6 Activity/Fragment lifecycle

Uma `Activity` passa por estados bem definidos — `onCreate` → `onStart` → `onResume` → `onPause` → `onStop` → `onDestroy` — que o sistema dispara conforme a tela fica visível, ganha foco, perde foco ou sai de cena. Um `Fragment` tem um ciclo ainda mais granular, porque sua `view` pode ser destruída e recriada (`onCreateView`/`onDestroyView`) sem que o `Fragment` inteiro seja destruído, e porque ele precisa se anexar/desanexar de uma `Activity` hospedeira (`onAttach`/`onDetach`).

Como `MainActivity` é uma `Activity` única 100% Compose, esse ciclo completo de `Fragment` não apareceria em nenhum outro lugar do projeto. Por isso existe `LifecycleDemoActivity` — uma `AppCompatActivity` clássica, com layout XML, que hospeda `LifecycleDemoFragment` — acessível pelo botão "Abrir demo de lifecycle" na tela principal:

```

1 class LifecycleDemoFragment : Fragment(R.layout.fragment_lifecycle_demo) {
2     private val entries = mutableListOf<String>()
3
4     override fun onAttach(context: Context) { super.onAttach(context); log("Fragment onAttach")
5         }
6     override fun onCreate(savedInstanceState: Bundle?) { super.onCreate(savedInstanceState); log
7         ("Fragment onCreate") }
8     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
9         super.onViewCreated(view, savedInstanceState); log("Fragment onViewCreated")
10    }
11    override fun onStart() { super.onStart(); log("Fragment onStart") }
12    override fun onResume() { super.onResume(); log("Fragment onResume") }
13    override fun onPause() { super.onPause(); log("Fragment onPause") }
14    override fun onStop() { super.onStop(); log("Fragment onStop") }
15    override fun onDestroyView() { log("Fragment onDestroyView"); super.onDestroyView() }
16    override fun onDetach() { super.onDetach(); log("Fragment onDetach") }
17
18    private fun log(message: String) {
19        Log.d("LifecycleDemo", message)
20        entries += message
21        view?.findViewById<TextView>(R.id.text_log)?.text = entries.joinToString("\n")
22    }
23 }

```

Listing 13: `Fragment`: cada callback grava e reimprime o log na tela

Rodando a demo, a ordem observada no Logcat (tag `LifecycleDemo`) é: `Activity.onCreate` → `Fragment.onAttach` → `Fragment.onCreate` → `Fragment.onViewCreated` → `Activity.onStart` → `Fragment.onStart` → `Activity.onResume` → `Fragment.onResume` — deixando claro que o `Fragment` está aninhado dentro do ciclo de vida da `Activity` que o hospeda, nunca à frente dele.

Mesmo em um app só-Compose, o `Lifecycle` continua relevante: o próprio `NotesCard` observa eventos de `ON_START`/`ON_STOP` da `Activity` para fins de log, usando a ponte oficial entre Compose e o sistema de `Lifecycle` clássico:

```

1 @Composable
2 private fun ObserveLifecycle(tag: String) {
3     val lifecycleOwner = LocalLifecycleOwner.current
4     DisposableEffect(lifecycleOwner) {
5         val observer = LifecycleEventObserver { _, event ->
6             if (event == Lifecycle.Event.ON_START || event == Lifecycle.Event.ON_STOP) {
7                 Log.d(tag, "Lifecycle event: $event")
8             }
9         }
10        lifecycleOwner.lifecycle.addObserver(observer)
11        onDispose { lifecycleOwner.lifecycle.removeObserver(observer) }
12    }
13 }

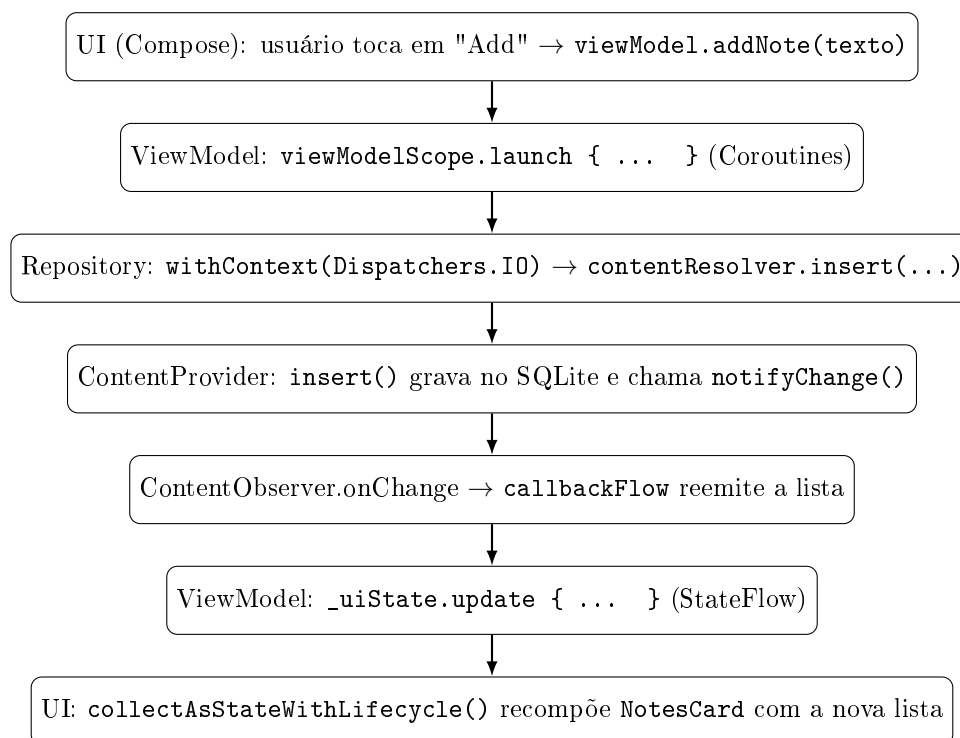
```

Listing 14: Observando o Lifecycle da Activity a partir de um Composable

DisposableEffect garante que o observador seja removido em **onDispose** quando o composable sai da árvore — sem isso, o observador vazaria e continuaria recebendo eventos de um Lifecycle cuja tela já não existe mais.

5 Fluxo de dados da tela Notas: do toque em "Add" até a UI atualizada

O diagrama abaixo conecta todos os tópicos da Parte II em um único ciclo — do toque no botão até a lista reaparecer atualizada:



Note que a única parte deste ciclo que *recompõe* algo é o último passo — todo o caminho até ali (ViewModel, Repository, Provider, SQLite) é Kotlin/Android comum, sem nenhuma dependência de Compose, o que é exatamente a separação de responsabilidades que o MVVM propõe.

6 Conclusão

O projeto **ComposeReview** cobre, em pouco código, sete tópicos centrais de um app Android moderno. Na Parte I, um contador e uma lista de tarefas revisam Jetpack Compose "puro": funções **@Composable** formam a árvore de UI; **state** (via **mutableStateOf** e **mutableStateListOf**) guarda os dados que podem mudar; **remember** preserva esse estado entre execuções; e a composição/recomposição mantém a tela sincronizada com o estado, seguindo o modelo declarativo **UI = f(state)**. Na Parte II, a tela Notas mostra como esses mesmos conceitos de Compose se encaixam em uma arquitetura maior: um **ContentProvider** compartilha dados persistidos em SQLite por trás de um contrato de URIs; **SharedPreferences** guarda preferências simples (com limitações claras para qualquer coisa maior que isso); **Coroutines** tiram o I/O da thread principal e transformam callbacks em **Flow**; o **ViewModel** organiza tudo em MVVM, expondo estado

via **StateFlow** (comparado a **LiveData**); e uma tela separada com Activity/Fragment clássicos deixa visível o ciclo de vida que, em um app só-Compose de Activity única, ficaria escondido.